

Detailed and Elaboratory Thesis chapter breakdown – MIS / CS / SE/ DS - UGC

Chapter – 01 [Introduction]

- 1.1 Chapter Overview. Briefly state what this chapter contains and how it introduces the research.
- 1.2 Problem Background - Explain the context and what has actually happened. Introduce the domain, current situation, affected users/systems, and evidence showing that the problem exists.
- 1.3 Problem Statement - State the specific problem clearly and support it with recent citations, statistics, or other applicable evidence. Focus on the problem rather than describing the proposed solution.
- 1.3.1 General Problem - Explain the high-level impact or implication caused by the problem.
- 1.3.2 Specific Problem - Explain the discipline/domain-specific shortcomings and conclude by highlighting the research gap that your project will address.
- 1.4 Research Question - Convert the core problem into a clear question that the study will answer. Prefer one main research question; relevant sub-questions may be included when necessary.
- 1.5 Research Motivation - Explain why this research is worth carrying out and what motivated the investigation.
- 1.6 Research Aim - State the overall outcome that you expect to achieve through the research.
- 1.7 Research Objectives - These must be specific to your research, not general learning objectives of the module. At undergraduate level, the following action-oriented phrases can be used to standardize the objectives:
 - 1.7.1 To identify
 - 1.7.2 To analyze
 - 1.7.3 To design / implement / develop
 - 1.7.4 To evaluate
- 1.8 Rich Picture of the Proposed Solution - Provide a high-level visual explanation of the proposed solution and its workflow. For computing projects, a system workflow, data pipeline, model workflow, or architecture-style diagram may be used when it communicates the solution more clearly.
- 1.9 Resource Requirements - Identify the resources required to complete the research. Include only resources that are relevant to the project.
- 1.9.1 Hardware - e.g., computers, servers, GPUs, sensors, mobile devices, or other physical resources, if required.
- 1.9.2 Software - e.g., programming languages, libraries, development tools, databases, cloud platforms, APIs, or licensed software, if required.
- 1.10 Project Scope - A small table with “In scope” and “Out of scope” may be included, supported by clear explanations of what the project will and will not cover.
- 1.11 Chapter Summary - Summarize the key points of the chapter and provide a short transition to the Literature Review.

2 Chapter - 02 [Literature Review] - A critical review is essential. Do not only summarize sources; compare, question, and discuss the literature in relation to your own research problem.

- 2.1 Chapter Overview - Explain how the literature review is organized and what areas will be reviewed.
- 2.2 Conceptual Map of the Literature - A visual map showing the main themes/concepts and how the reviewed literature is organized. This helps the reader understand the structure of the chapter quickly.
- 2.3 Domain Overview - Provide surface-level but significant background information about the research domain - 10 % coverage
- 2.4 Existing Systems / Frameworks / Designs - Compare relevant existing solutions, systems, models, methods, or frameworks and support the comparison with your own critical analysis - 30 % coverage
- 2.5 Technological Analysis - Analyze the technologies, methods, models, algorithms, or technical approaches that are relevant to the proposed research - 60% coverage
 - 2.5.1 Algorithmic analysis - Compare relevant algorithms/models, their assumptions, strengths, limitations, performance, and suitability for your problem.
 - 2.5.2 Design analysis - Compare relevant system architectures, model designs, data structures, interfaces, or design approaches, depending on the project type.
 - 2.5.3 Workflow analysis - Compare how existing solutions process data or execute tasks from input to output, and identify workflow limitations relevant to your research.
- 2.6 Reflection - Synthesize the reviewed literature and justify the research gap. Use mainly recent literature (normally from the most recent 5 years); older sources may be used when they are foundational works, established theories, or theorems.

Whether an existing approach is suitable or unsuitable for the current problem, justify the decision using evidence from the reviewed literature. **Every major technical or design decision should have a clear and evidence-based rationale.**

3 Chapter - 03 [Methodology] - Discuss with your supervisor and use a methodology that matches the nature of your computing research.

- 3.1 Research Paradigm - Explain the overall research philosophy/worldview guiding the study (for example, positivism, interpretivism, or pragmatism). For strongly technical or experimental computing projects, keep this concise and explain why the chosen paradigm fits the evidence you will collect and analyze.
- 3.2 Research Approach - Explain the reasoning approach used, such as deductive, inductive, or abductive, as appropriate. Do not confuse the research approach with the research paradigm.

- 3.3 Research Strategy - State the practical research strategy used to answer the research question, e.g., experiment, design science, case study, survey, simulation, prototype development, or machine-learning/model development and evaluation, as applicable.
 - 3.4 Data Collection / Data Sources - Explain where the data or evidence comes from and how it is obtained. This may include public/secondary datasets, APIs, system logs, sensors, experiments, questionnaires, interviews, or other suitable sources. Also describe selection criteria and permissions where relevant.
 - 3.5 Research Methodology Execution Workflow - A table or workflow diagram may be used to show how the following stages are carried out in your research and how they connect to the objectives.
 - 3.5.1 Problem Identification - Define the problem using evidence and the research context.
 - 3.5.2 Relevance Justification - Explain why solving the problem is important and timely.
 - 3.5.3 Comparative Analysis and Gap Justification - Use prior work to show what is missing or insufficient.
 - 3.5.4 Define and Finalize Objectives - Link each objective directly to the research problem and planned evaluation.
 - 3.5.5 Design / Development / Data Management and Handling - Explain the planned system/model/algorithm design, data preparation, storage, preprocessing, and development steps as relevant.
 - 3.5.6 Evaluation and Communication - Explain how the proposed solution will be evaluated and how the findings will be reported.
 - 3.6 Project Management Methodology - Justify a suitable method based on the type and scale of the project. SCRUM, KANBAN, PRINCE 2, an iterative/agile approach, or another appropriate method may be used. For an individual research project, a simple milestone-based or iterative plan can also be acceptable if justified.
 - 3.6.1 Project Timeline - Present the major research/development activities and their planned time periods, e.g., using a Gantt chart.
 - 3.6.2 Ethical Considerations - Discuss applicable issues such as informed consent, privacy, personal/sensitive data, cybersecurity, fairness/bias, responsible AI, intellectual property, dataset/software licenses, and secure data handling.
 - 3.7 Chapter Summary - Summarize the selected methodology and explain how it supports the research objectives.
- 4 System / Model Requirement Specification** - For software/system-development projects, use an SRS-style structure. For AI, data science, algorithmic, or experimental projects, adapt this chapter to describe the relevant data, model, functional, performance, and experimental requirements.
- 4.1 Chapter Overview - State what requirements, analyses, and design specifications are presented in this chapter.
 - 4.2 Stakeholder Analysis - Identify the people or organizations affected by or interested in the solution (e.g., users, clients, administrators, domain experts, data owners, or

decision-makers). If the project has no direct end user, identify the intended beneficiaries and research stakeholders.

- 4.3 Operationalization Process - Show how the research objectives are converted into measurable or implementable items, such as questionnaire questions, dataset variables/features, interview themes, evaluation metrics, system requirements, or experimental measures. Explain how the collected evidence validates the selected requirements.
- 4.4 System / Model Analysis - Analyze the proposed solution before implementation. Use only the representations that are meaningful for your project type.
 - 4.4.1 Use Case Diagrams with specifications - Mainly for software/system projects. Include only the most important specs and push others to the appendix to save the page count.
 - 4.4.2 Class Diagram - Use for object-oriented/system projects when it meaningfully explains the software structure. Otherwise, use a more suitable representation with supervisor approval.
 - 4.4.3 Activity Diagram - Use to show important process/workflow logic where applicable; a data-processing or experiment-flow diagram may be used for AI/DS projects when more suitable.
 - 4.4.4 Sequence diagrams mapped with use cases - Mainly for software/system projects; max 03 is enough. For non-software projects, include only if interaction sequences are relevant.
 - 4.4.5 State chart or Deployment diagrams as per the nature of the project (optional).
- 4.5 Proposed system architecture / framework / model architecture diagram as necessary. For AI/DS projects, this can show the data pipeline, preprocessing, training/inference stages, model components, and outputs.
- 4.6 Functional and Non-functional Requirements - For software systems, define required functions and quality attributes (e.g., performance, security, usability, reliability). For AI/DS projects, adapt this to data, model, performance, explainability, reproducibility, or operational requirements as relevant.
- 4.7 Chapter Summary.

5 Implementation / Designing

- 5.1 Elaborate in detail on the steps followed to create the system/framework/model or to design the algorithms. For AI/DS work, include relevant data preparation, feature engineering, model training, tuning, and inference steps as applicable.
 - 5.1.1 Provide pseudocode / flow charts / algorithms / models as necessary to explain the core logic and your contribution.
 - 5.1.2 If it is a framework or pipeline, explain the workflow using a block-diagram series as necessary.
 - 5.1.3 Justify the selection of technologies (a tabular reflection would be adequate). Explain why each selected technology is suitable rather than simply listing it.

- 5.1.3.1 Programming Language
 - 5.1.3.2 Libraries / Packages Used
 - 5.1.3.3 Backend and front-end frameworks used (if applicable). For non-web projects, replace this with relevant databases, ML frameworks, cloud platforms, simulation tools, or other development environments.
 - 5.2 Selected significantly important implementation attempts should be explained using UI designs, code snippets, model/pipeline evidence, outputs, or execution evidence as appropriate. Focus on contribution-associated features and non-trivial logic; do not spend space on basic features such as login/sign-up unless they contain research-specific logic introduced by you.
 - 5.3 Chapter Summary
- 6 Testing and Evaluation - Justify why the selected testing/evaluation strategies are appropriate for the research instead of including tests without a reason. Present outcomes using numerical results, tables, and graphs where appropriate.
 - 6.1 Chapter Overview - Explain what will be tested/evaluated, the criteria used, and how the results will answer the research objectives.
 - 6.2 Test Plan and Test Cases (max 10, push rest to appendix) - Mainly for software/system testing. For AI/DS/algorithmic projects, the equivalent evaluation experiments/cases may be described here as appropriate, while keeping the stated limit when presented as test cases.
 - 6.2.1 Nonfunctional Testing - e.g., performance, scalability, security, reliability, usability, maintainability, or efficiency, depending on the system.
 - 6.2.2 Functional Testing - Verify that the implemented functions/features produce the expected behavior and outputs.
 - 6.3 Testing / Evaluation Workflow - Show the sequence from test/experiment preparation to execution, measurement, analysis, and interpretation.
 - 6.4 Review the testing/evaluation strategies that are applicable to your research. Not every method is suitable for every project. Software projects may use functional/non-functional, unit, integration, system, usability, or performance testing. AI/DS projects may use train/validation/test separation, suitable performance metrics, cross-validation, baseline comparison, error analysis, robustness checks, or ablation studies. Algorithmic projects may use correctness tests, complexity analysis, benchmarking, or scalability experiments. Select only what is relevant and justify it.
- 7 Concluding Remarks
 - 7.1 Accomplishment of the Research Objectives - Address each objective explicitly and support the conclusion with evidence from the Testing and Evaluation chapter. Use triangulation only when the study genuinely combines multiple methods, data sources, evaluators, or forms of evidence.

- 7.2 Problems Encountered - Briefly explain important technical, data, methodological, or project-management challenges and how they were handled. Distinguish these from research limitations where appropriate.
- 7.3 Self-reflection - Reflect on your own research/development decisions, learning, strengths, and areas for improvement; avoid a generic personal narrative.
 - 7.3.1 Your reflection on the research carried out and the key decisions you made.
 - 7.3.2 Benefits / skills gained from carrying out the research.
 - 7.3.3 Learning curves - Explain important learning challenges and how your understanding or technical capability developed during the project.
- 7.4 Business / Practical Insight of Your Idea - Discuss the practical value of the work. Depending on the project, this may include industry, organizational, social, scientific, operational, or commercial value.
 - 7.4.1 Real-world application possibilities of your concept / idea, including likely users/beneficiaries and realistic deployment or adoption considerations.
- 7.5 Future Recommendations - Propose realistic future improvements or research directions based on the current results, limitations, and unresolved issues.